

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software

**Implementación y Verificación de un Clúster de
Base de Datos
Distribuida con CockroachDB Aplicado al Sistema
de
Gestión de Soporte Técnico ISP (Equipo ACC)**

Código de actividad: GA-SUM-03 / PE-U3

Asignatura: Aplicaciones Distribuidas (ISR-701)

Unidad: 3 – Bases de Datos Distribuidas

Tipo de guía: Práctica Experimental

Modalidad: Grupal (Sumativa)

Período académico: 2026–2027 PPA

Docente: Gleiston Cicerón Guerrero Ulloa, M.Sc.

PFC de referencia: ACC – Soporte Técnico ISP

Sistema: Gestión de Tickets de Soporte Técnico

Integrante A Ernesto Gregory Luna Mora

Integrante B Cristhian Daniel Pacheco Cárdenas

Integrante C Robinson Rodrigo Cando Moreno

Justificación de la elección del PFC (máx. 100 palabras):

El dominio ACC – Soporte Técnico ISP gestiona tickets de atención distribuidos naturalmente en tres zonas geográficas operativas (Norte, Centro, Sur), lo que ofrece una clave de fragmentación horizontal clara y con sentido de negocio real, a diferencia de un particionamiento artificial. Esta segmentación regional permite ejercitar directamente los conceptos de fragmentación horizontal por lista y de localidad de datos estudiados en la unidad. Asimismo, al tratarse de un servicio de atención al cliente que no puede interrumpirse, el dominio justifica plenamente la necesidad de replicación y tolerancia a fallos en un clúster de tres nodos, evidenciando de forma concreta las garantías de disponibilidad que ofrece el algoritmo de consenso Raft.

Repositorio del proyecto: <https://github.com/Ernesto835/SOPORTE-ISP.git>

Índice

Resumen	2
Introducción	2
1. Fundamento Teórico	3
1.1. Fundamentos y Arquitecturas de Bases de Datos Distribuidas	3
1.2. Fragmentación y Replicación de Datos	4
1.3. Control de Concurrencia Distribuido y Consenso	6
1.4. Consistencia, Disponibilidad, PACELC y Sistemas NewSQL/NoSQL . .	7
2. Procedimiento Aplicado	9
2.1. Levantamiento del Clúster CockroachDB	9
2.2. Implementación del Esquema del PFC	12
2.2.1. Fragmentación horizontal aplicada	14
2.3. Verificación de Tolerancia a Fallos	15
2.4. Medición de Rendimiento	17
3. Resultados	18
3.1. Resultados de la implementación del clúster	19
3.2. Resultados del esquema y la fragmentación	19
3.3. Resultados de la prueba de tolerancia a fallos	19
3.4. Resultados de la medición de rendimiento	19
4. Análisis Crítico	19
5. Preguntas de Análisis	21
6. Conclusiones	24
6.1. Conclusión Grupal	24
6.2. Conclusiones Individuales	25
7. Declaración de Uso de Inteligencia Artificial Generativa	26

Resumen

El presente informe documenta la implementación, configuración y verificación de un clúster de base de datos distribuida CockroachDB de tres nodos, aplicado al esquema de datos del dominio ACC – Soporte Técnico ISP, en el marco de la Práctica Experimental GA-SUM-03/PE-U3 de la asignatura Aplicaciones Distribuidas. Se implementó una estrategia de fragmentación horizontal por región geográfica sobre la tabla principal de tickets, verificada tanto a nivel lógico como físico mediante los comandos nativos de CockroachDB. Se realizó una prueba experimental de tolerancia a fallos, deteniendo un nodo del clúster y verificando que el sistema mantuvo el servicio de lectura y escritura gracias al quórum del algoritmo de consenso Raft, reincorporando posteriormente el nodo en un tiempo de 2.0 segundos. Adicionalmente, se ejecutó una comparación cuantitativa de rendimiento entre el clúster distribuido y una instancia de nodo único sobre cinco consultas representativas del dominio, evidenciando un patrón mixto de resultados según la naturaleza de coordinación requerida por cada consulta. El proyecto integra fundamento teórico verificado bibliográficamente, evidencia experimental reproducible y análisis crítico de los resultados obtenidos, cumpliendo los objetivos de aprendizaje de la Unidad 3 de la asignatura.

Introducción

Las bases de datos distribuidas constituyen un componente central de la infraestructura de sistemas modernos que requieren alta disponibilidad, escalabilidad horizontal y tolerancia a fallos. Sistemas como CockroachDB implementan estas propiedades mediante técnicas de fragmentación, replicación y algoritmos de consenso distribuido, cuya comprensión teórica y verificación práctica constituyen el objetivo central de la Unidad 3 de la asignatura Aplicaciones Distribuidas.

El presente trabajo tiene como objetivo general implementar, configurar y verificar un clúster funcional de base de datos distribuida de tres nodos aplicado al esquema de datos del Proyecto Fin de Curso (PFC) de referencia, demostrando transparencia de fragmentación y replicación, comportamiento ante fallos de nodos y recuperación automática mediante consenso Raft. Como objetivos específicos, se busca: (1) configurar un clúster CockroachDB de tres nodos en contenedores Docker con replicación automática; (2) implementar el esquema de datos del PFC con la estrategia de fragmentación diseñada previamente; (3) demostrar experimentalmente la tolerancia a fallos; (4) medir y comparar cuantitativamente el rendimiento distribuido frente a un nodo único; y (5) redactar un documento académico que integre fundamento teórico, evidencia experimental y análisis crítico.

El equipo seleccionó como dominio de referencia el PFC ACC – Soporte Técnico ISP, un sistema de gestión de tickets de soporte técnico para un proveedor de servicios de internet, cuyo modelo de datos –compuesto por clientes, tickets y un historial de

auditoría de estados— permite ejercitar de forma representativa los conceptos de fragmentación horizontal por región geográfica y las garantías de consistencia transaccional que ofrece un sistema distribuido serializable.

El documento se estructura de la siguiente manera: la Sección 1 desarrolla el fundamento teórico correspondiente a los cuatro subtemas de la Unidad 3; la Sección 2 describe el procedimiento experimental aplicado; las Secciones 3 y 4 presentan y analizan los resultados obtenidos; la Sección 5 responde las preguntas de análisis obligatorias; y finalmente las Secciones 6 y 7 presentan las conclusiones del equipo y la declaración de uso de herramientas de inteligencia artificial generativa.

1. Fundamento Teórico

1.1. Fundamentos y Arquitecturas de Bases de Datos Distribuidas

Una base de datos distribuida (BDD) se define como una colección de múltiples bases de datos lógicamente interrelacionadas, distribuidas físicamente sobre una red de computadoras, de modo que para el usuario final el sistema se comporta como si fuera una única base de datos centralizada [1]. El sistema de software que administra esta colección y hace transparente su distribución ante las aplicaciones se denomina sistema de gestión de bases de datos distribuidas (SGBDD) [1]. Esta distinción es fundamental: mientras la BDD es el conjunto de datos físicamente repartidos, el SGBDD es el motor que coordina el acceso, la actualización y la consistencia de dichos datos sin que el usuario deba conocer su ubicación real [2].

El diseño de un SGBDD se orienta tradicionalmente por los doce objetivos formulados por C.J. Date, que buscan que un sistema distribuido se comporte, desde la perspectiva del usuario, de forma equivalente a un sistema centralizado. Entre estos objetivos se destacan: la autonomía local de cada nodo, la ausencia de dependencia de un sitio central, la operación continua del sistema, la independencia respecto a la ubicación de los datos, la independencia de fragmentación, la independencia de replicación, el procesamiento distribuido de consultas, la gestión distribuida de transacciones, la independencia del hardware, la independencia del sistema operativo, la independencia de la red de comunicaciones y la independencia del SGBD subyacente [1]. En conjunto, estos objetivos plantean un ideal de transparencia casi total, aunque en la práctica los sistemas reales, como CockroachDB, priorizan un subconjunto de ellos según sus casos de uso.

Nota de verificación: la lista anterior de los doce objetivos de Date corresponde a la versión estándar más citada en la literatura académica; se recomienda contrastarla contra la edición específica de Özsu y Valduriez disponible para el equipo antes de la entrega final.

En cuanto a las arquitecturas de referencia, la literatura distingue principalmente tres modelos organizativos. La arquitectura cliente-servidor separa la lógica de

gestión de datos (servidor) de la lógica de aplicación (cliente), permitiendo que varios clientes accedan a uno o más servidores especializados [1]. La arquitectura colaborativa (peer-to-peer) elimina la distinción rígida entre cliente y servidor: cada nodo puede actuar simultáneamente como proveedor y consumidor de datos, un patrón que adoptan sistemas modernos como CockroachDB, donde cualquier nodo puede recibir y coordinar una consulta [1]. Finalmente, la arquitectura basada en middleware introduce una capa intermedia que abstrae la heterogeneidad de los sistemas subyacentes, facilitando la integración de fuentes de datos diversas [2].

Un concepto central en el diseño de BDD es la transparencia, entendida como la ocultación de los detalles de implementación distribuida ante el usuario o la aplicación. Se reconocen ocho formas de transparencia: la transparencia de fragmentación, que oculta cómo se dividen las tablas; la transparencia de replicación, que oculta la existencia de copias redundantes; la transparencia de ubicación, que evita que el usuario conozca el nodo físico donde residen los datos; la transparencia de acceso, que unifica la interfaz de consulta independientemente del origen de los datos; la transparencia de concurrencia, que garantiza que las transacciones concurrentes no interfieran entre sí de forma visible; la transparencia de fallos, que permite que el sistema continúe operando ante la caída de un nodo; la transparencia de escalabilidad, que permite añadir nodos sin afectar las aplicaciones existentes; y la transparencia de evolución, que permite modificar el esquema sin interrumpir el servicio [1].

Por último, es necesario distinguir entre BDD homogéneas, donde todos los nodos ejecutan el mismo SGBD y comparten un esquema común, y BDD heterogéneas, donde coexisten distintos motores de bases de datos y modelos de datos, lo que exige mecanismos de traducción y mediación adicionales [1]. En el contexto del presente proyecto, el clúster CockroachDB implementado constituye una BDD homogénea de tipo colaborativo, donde los tres nodos ejecutan el mismo motor y cooperan bajo el algoritmo de consenso Raft. La aplicación al dominio del PFC de Soporte Técnico ISP resulta pertinente porque el sistema de gestión de tickets requiere transparencia de fragmentación (los tickets se dividen por región) y transparencia de fallos (el servicio de soporte no puede interrumpirse ante la caída de un nodo), ejercitando así los principios fundamentales expuestos en esta sección.

1.2. Fragmentación y Replicación de Datos

La fragmentación es el proceso mediante el cual una relación global R se divide en subconjuntos más pequeños, denominados fragmentos, que se distribuyen físicamente entre los distintos nodos del sistema [1]. Para que una fragmentación sea correcta desde el punto de vista formal, debe satisfacer tres condiciones fundamentales. La condición de completitud exige que todo dato presente en la relación original pueda reconstruirse a partir de sus fragmentos, sin pérdida de información [1]. La condición de reconstrucción garantiza que, mediante un operador relacional adecuado, sea posi-

ble reconstruir la relación original R a partir de la unión (en el caso horizontal) o de la combinación por junta (en el caso vertical) de sus fragmentos [1]. Finalmente, la condición de disjunción establece que, en la fragmentación horizontal, los fragmentos no deben compartir tuplas entre sí, evitando así redundancia no controlada [1].

Formalmente, la fragmentación horizontal primaria de una relación R mediante un predicado p_i se expresa utilizando el operador de selección σ del álgebra relacional:

$$R_i = \sigma_{p_i}(R) \quad (1)$$

cumpléndose las condiciones de completitud y disjunción:

$$\bigcup_{i=1}^n R_i = R \quad (\text{completitud}) \quad (2)$$

$$R_i \cap R_j = \emptyset \quad \forall i \neq j \quad (\text{disjunción}) \quad (3)$$

En el caso de la fragmentación horizontal derivada, un fragmento de una relación dependiente se define en función de la fragmentación ya realizada sobre una relación propietaria, mediante el operador de semi-junta $\bowtie$:

$$R_i = R \bowtie R'_i \quad (4)$$

donde R'_i representa el fragmento correspondiente de la relación propietaria [1].

La fragmentación vertical, por su parte, divide una relación en subconjuntos de atributos en lugar de tuplas, agrupando las columnas según su afinidad de uso: aquellos atributos que suelen consultarse conjuntamente en las mismas transacciones se agrupan en un mismo fragmento, reduciendo así la necesidad de operaciones de junta costosas entre nodos [1]. Cuando una relación no puede fragmentarse adecuadamente mediante un único criterio, se recurre a la fragmentación mixta o híbrida, que combina fragmentación horizontal y vertical de forma sucesiva, ajustándose a patrones de acceso más complejos [1].

En cuanto a la replicación, esta consiste en el mantenimiento de copias idénticas de un fragmento en más de un nodo del sistema, con el objetivo de aumentar la disponibilidad y tolerancia a fallos [2]. La replicación síncrona exige que todas las réplicas confirmen la escritura antes de que esta se considere completada, garantizando consistencia fuerte a costa de mayor latencia, especialmente en entornos geodistribuidos [3]. La replicación asíncrona, en cambio, confirma la escritura tan pronto como la réplica primaria la procesa, propagando el cambio a las demás réplicas en segundo plano; esto reduce la latencia percibida por el cliente, pero introduce una ventana temporal de inconsistencia entre réplicas [3]. CockroachDB adopta un enfoque de replicación síncrona basada en el algoritmo de consenso Raft, donde una escritura solo se confirma una vez que la mayoría (quórum) de las réplicas del rango correspondiente ha registrado la

entrada en su log, lo cual constituye un compromiso deliberado entre consistencia y disponibilidad [4].

En el contexto del PFC de Soporte Técnico ISP, la estrategia de fragmentación horizontal aplicada a la tabla `tickets` mediante el atributo `region` resulta directamente justificable con los conceptos anteriores: cada fragmento R_i corresponde a los tickets de una región geográfica específica (norte, centro, sur), cumpliendo la condición de disjunción al no existir solapamiento entre regiones, y la condición de completitud al garantizar que la unión de los tres fragmentos reconstruye la totalidad de los tickets del sistema.

1.3. Control de Concurrency Distribuido y Consenso

El control de concurrencia en bases de datos distribuidas aborda el problema de garantizar que múltiples transacciones que se ejecutan simultáneamente sobre distintos nodos no produzcan resultados inconsistentes, preservando las propiedades ACID a pesar de la ausencia de un reloj global compartido [1]. A diferencia de un sistema centralizado, en un entorno distribuido cada nodo puede poseer una visión parcial del estado del sistema, lo que complica la detección de conflictos entre transacciones concurrentes [5].

Uno de los mecanismos clásicos para resolver este problema es el protocolo de bloqueo en dos fases (2PL), en el cual una transacción atraviesa una fase de crecimiento, durante la cual únicamente adquiere bloqueos, y una fase de decrecimiento, durante la cual únicamente los libera, garantizando serializabilidad [1]. Su variante estricta, el protocolo S2PL (Strict 2PL), exige además que todos los bloqueos de escritura se mantengan hasta el momento del compromiso (*commit*) de la transacción, evitando lecturas sucias y facilitando la recuperación ante fallos [1]. En un entorno distribuido, la aplicación de 2PL requiere coordinación entre los nodos que participan en una transacción, lo cual incrementa la latencia y el riesgo de interbloqueos (*deadlocks*) distribuidos.

La detección de interbloqueos distribuidos se realiza habitualmente mediante grafos de espera (*wait-for graphs*), donde cada nodo del grafo representa una transacción y cada arista indica que una transacción espera la liberación de un recurso bloqueado por otra [1]. Cuando este grafo, construido de forma distribuida a partir de la información local de cada nodo, contiene un ciclo, se confirma la existencia de un interbloqueo que debe resolverse abortando alguna de las transacciones involucradas [5].

Para garantizar la atomicidad de transacciones que abarcan múltiples nodos, se emplea el protocolo de compromiso en dos fases (2PC). En la primera fase (*voting phase*), un coordinador solicita a todos los participantes que confirmen si están en condiciones de comprometer la transacción; en la segunda fase (*commit phase*), si todos los participantes responden afirmativamente, el coordinador ordena el compromiso definitivo, y en caso contrario, ordena su aborto [1]. El principal punto débil de

2PC es su vulnerabilidad ante la caída del coordinador tras la fase de votación: los participantes que ya votaron afirmativamente quedan bloqueados, sin poder decidir de forma autónoma si comprometer o abortar, hasta que el coordinador se recupere [1]. El protocolo de compromiso en tres fases (3PC) introduce una fase intermedia adicional (*pre-commit*) que reduce, aunque no elimina completamente, este riesgo de bloqueo indefinido, a costa de mayor latencia y complejidad [5].

Frente a las limitaciones de 2PC y 3PC en presencia de fallos del coordinador, los sistemas distribuidos modernos como CockroachDB recurren a algoritmos de consenso como Raft, diseñado explícitamente para ser comprensible y verificable [6]. Raft organiza el clúster en torno a un único líder elegido por mayoría de votos entre los nodos disponibles; todas las escrituras se dirigen a este líder, quien las registra en su log y las replica a los nodos seguidores mediante mensajes `AppendEntries` [6]. Una entrada de log se considera comprometida únicamente cuando ha sido replicada en la mayoría (quórum) de los nodos del clúster, lo que garantiza que, incluso ante la caída de una minoría de nodos, el sistema conserve un registro consistente y pueda continuar operando [6]. Si el líder deja de enviar señales de vida (*heartbeats*) dentro de un intervalo de tiempo determinado, los seguidores inician un proceso de elección mediante mensajes `RequestVote`, postulándose como candidatos hasta que uno de ellos obtenga la mayoría de votos y asuma el liderazgo [6]. CockroachDB implementa este mecanismo a nivel de cada rango (*range*) de datos, de modo que distintos rangos pueden tener líderes distintos distribuidos entre los nodos del clúster [4].

En el experimento realizado sobre el PFC de Soporte Técnico ISP, este marco teórico se verifica directamente: al detener el nodo `crdb-node2`, el líder del rango correspondiente a la tabla `tickets` conserva el quórum con los dos nodos restantes, permitiendo que las consultas continúen respondiendo correctamente sin necesidad de una nueva elección de líder, dado que el nodo caído era un seguidor y no el líder del rango afectado.

1.4. Consistencia, Disponibilidad, PACELC y Sistemas NewSQL/NoSQL

El teorema CAP, formalizado por Gilbert y Lynch a partir de la conjetura original de Brewer, establece que un sistema de almacenamiento distribuido no puede garantizar simultáneamente las tres propiedades de consistencia (*Consistency*), disponibilidad (*Availability*) y tolerancia a particiones (*Partition tolerance*); ante la ocurrencia de una partición de red, el sistema debe sacrificar necesariamente consistencia o disponibilidad [7]. Esta formalización demuestra matemáticamente, mediante un modelo de sistema asíncrono con al menos dos nodos, que ningún algoritmo puede satisfacer las tres garantías de forma simultánea cuando existen mensajes que pueden perderse [7].

Doce años después de su conjetura original, Brewer revisó y matizó su planteamiento, señalando que la disyuntiva entre consistencia y disponibilidad no es absoluta ni permanente, sino que ocurre únicamente durante el intervalo en que la partición

está activa, y que los sistemas reales pueden aplicar estrategias de mitigación, como la degradación parcial del servicio o la reconciliación posterior al restablecimiento de la comunicación [8]. Esta reinterpretación es relevante porque evita la lectura simplista de CAP como una elección binaria y permanente, y reconoce que un mismo sistema puede comportarse de forma distinta ante distintos tipos de operación.

Ante las limitaciones de CAP para describir el comportamiento de un sistema en ausencia de particiones, Abadi propuso el modelo PACELC, que extiende el análisis señalando que, incluso cuando la red opera con normalidad (*Else*), todo sistema distribuido debe elegir entre latencia (*Latency*) y consistencia (*Consistency*), además de la disyuntiva entre disponibilidad y consistencia durante una partición (*Partition*) [9]. Este modelo resulta particularmente relevante para bases de datos geodistribuidas, donde la latencia de red entre regiones geográficas impone una tensión constante entre la velocidad de respuesta y el nivel de consistencia garantizado, incluso en condiciones normales de operación [9].

El espectro de modelos de consistencia en sistemas de almacenamiento no transaccionales es amplio y no se limita a la dicotomía entre consistencia fuerte y consistencia eventual. Viotti y Vukolić catalogan sistemáticamente decenas de modelos intermedios, como la consistencia causal, la consistencia de sesión y la consistencia acotada por tiempo, cada uno con distintas garantías sobre el orden y la visibilidad de las operaciones de escritura entre réplicas [3]. En este espectro, CockroachDB se ubica en el extremo de mayor garantía, al implementar un nivel de aislamiento serializable mediante un protocolo de concurrencia optimista combinado con relojes híbridos, lo que le permite ofrecer consistencia fuerte equivalente a la de un sistema centralizado, incluso operando de forma distribuida [4].

En contraste, los sistemas NoSQL adoptan con frecuencia modelos de consistencia más relajados en favor de la disponibilidad y el rendimiento. Dynamo, el almacén clave-valor desarrollado por Amazon, prioriza la disponibilidad mediante replicación con *quorum* ajustable y resolución de conflictos en el lado del cliente, aceptando consistencia eventual como compromiso de diseño [10]. Spanner, por su parte, representa un caso híbrido dentro de la categoría NewSQL: combina un modelo relacional con distribución global y logra consistencia externa (*external consistency*) mediante el uso de relojes atómicos y GPS sincronizados con una cota de incertidumbre acotada (TrueTime) [11]. Apache Cassandra, un sistema de almacenamiento columnar descentralizado, adopta un modelo de consistencia eventual ajustable por operación, permitiendo al desarrollador equilibrar consistencia y disponibilidad según el nivel de consistencia (*consistency level*) configurado en cada lectura o escritura [12]. MongoDB, como base de datos orientada a documentos, ofrece un modelo flexible que permite tanto lecturas eventualmente consistentes como transacciones multidocumento con garantías ACID, dependiendo de la configuración del nivel de *write concern* y *read concern* establecido [13].

La comparación entre CockroachDB, MongoDB y Cassandra en el eje consistencia/disponibilidad/latencia (C/A/L) evidencia posicionamientos claramente diferenciados dentro del espectro PACELC. CockroachDB se orienta hacia PC/EC: prioriza consistencia tanto durante una partición como en operación normal, asumiendo mayor latencia como costo [4]. Cassandra se orienta hacia PA/EL: prioriza disponibilidad y baja latencia tanto durante una partición como en operación normal, relegando la consistencia fuerte a una opción configurable [12]. MongoDB ocupa una posición intermedia, dado que su comportamiento por defecto favorece la disponibilidad, pero permite configurar niveles de consistencia más estrictos a costa de latencia adicional, aproximándose a PA/EC según la configuración elegida [13].

Tabla 1: Comparación de CockroachDB, MongoDB y Cassandra según el modelo PACELC

Criterio	CockroachDB	MongoDB	Cassandra
Modelo de datos	Relacional (NewSQL)	Documental (NoSQL)	Columnar (NoSQL)
Consistencia (partición)	Consistente (PC)	Configurable	Disponible (PA)
Consistencia (normal)	Consistente (EC)	Configurable	Baja latencia (EL)
Clasificación PACELC	PC/EC	PA/EC (config.)	PA/EL
Aislamiento	Serializable	Configurable	Eventual (ajustable)
Referencia	[4]	[13]	[12]

Para el dominio del PFC de Soporte Técnico ISP, esta discusión resulta directamente aplicable: un sistema de gestión de tickets que registra el estado de atención al cliente requiere consistencia fuerte para evitar que dos agentes de soporte modifiquen simultáneamente el mismo ticket con información contradictoria, lo que justifica la elección de CockroachDB frente a alternativas orientadas a disponibilidad como Cassandra.

2. Procedimiento Aplicado

En esta sección se documenta la implementación práctica del clúster de base de datos distribuida CockroachDB, aplicado al esquema de datos del dominio ACC – Soporte Técnico ISP, siguiendo la metodología establecida en la guía de práctica GA-SUM-03/PE-U3.

2.1. Levantamiento del Clúster CockroachDB

El clúster se implementó mediante contenedores Docker orquestados con Docker Compose, definiendo tres nodos CockroachDB interconectados por una red de tipo *bridge* denominada **roach-net**, además de una cuarta instancia de nodo único destinada exclusivamente a la comparación de rendimiento del Paso 4. Cada nodo expone un puerto SQL individual (26257, 26258 y 26259 respectivamente) y un puerto de consola web (8080, 8081 y 8082), permitiendo el acceso independiente a cada réplica durante

las pruebas. La configuración incluye un límite de memoria de 512 MB por contenedor (`mem_limit`), verificaciones de estado (`healthcheck`) mediante solicitudes HTTP al endpoint de salud de cada nodo, y volúmenes persistentes individuales que preservan los datos entre reinicios del contenedor.

Listing 1: Configuración de Docker Compose para el clúster de 3 nodos CockroachDB

```
version: '3.8'

networks:
  roach-net:
    driver: bridge

services:
  crdb-node1:
    image: cockroachdb/cockroach:v24.3.0
    container_name: crdb-node1
    hostname: crdb-node1
    networks:
      - roach-net
    ports:
      - "26257:26257"
      - "8080:8080"
    volumes:
      - node1-data:/cockroach/cockroach-data
    command: start --insecure --join=crdb-node1,crdb-node2,crdb-node3
      --advertise-addr=crdb-node1
    mem_limit: 512m
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 10s
      timeout: 5s
      retries: 3

  crdb-node2:
    image: cockroachdb/cockroach:v24.3.0
    container_name: crdb-node2
    hostname: crdb-node2
    networks:
      - roach-net
    ports:
      - "26258:26257"
      - "8081:8080"
    volumes:
      - node2-data:/cockroach/cockroach-data
    command: start --insecure --join=crdb-node1,crdb-node2,crdb-node3
      --advertise-addr=crdb-node2
    mem_limit: 512m
    healthcheck:
```

```

    test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
    interval: 10s
    timeout: 5s
    retries: 3

crdb-node3:
  image: cockroachdb/cockroach:v24.3.0
  container_name: crdb-node3
  hostname: crdb-node3
  networks:
    - roach-net
  ports:
    - "26259:26257"
    - "8082:8080"
  volumes:
    - node3-data:/cockroach/cockroach-data
  command: start --insecure --join=crdb-node1,crdb-node2,crdb-node3
    --advertise-addr=crdb-node3
  mem_limit: 512m
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
    interval: 10s
    timeout: 5s
    retries: 3

cockroach-single:
  image: cockroachdb/cockroach:v24.3.0
  container_name: cockroach-single
  hostname: cockroach-single
  networks:
    - roach-net
  ports:
    - "26260:26257"
    - "8083:8080"
  volumes:
    - single-data:/cockroach/cockroach-data
  command: start-single-node --insecure --advertise-addr=cockroach-
    single
  mem_limit: 512m
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
    interval: 10s
    timeout: 5s
    retries: 3

volumes:
  node1-data:
  node2-data:

```

```
node3-data:
single-data:
```

Una vez levantados los contenedores mediante `docker compose up -d`, el clúster de tres nodos se inicializó ejecutando `cockroach init` desde el primer nodo, y su estado se verificó mediante el comando `node status`, confirmando que los tres nodos reportaran `is_live=true`. La Figura 1 muestra el panel de control (*Dashboard*) de CockroachDB accedido en `http://localhost:8080`, evidenciando los tres nodos activos ejecutando la versión v24.3.0, con licencia Enterprise Free activa (359 días de vigencia restante al momento de la captura) y una distribución saludable de 55 rangos totales, sin rangos infrarreplicados ni no disponibles.

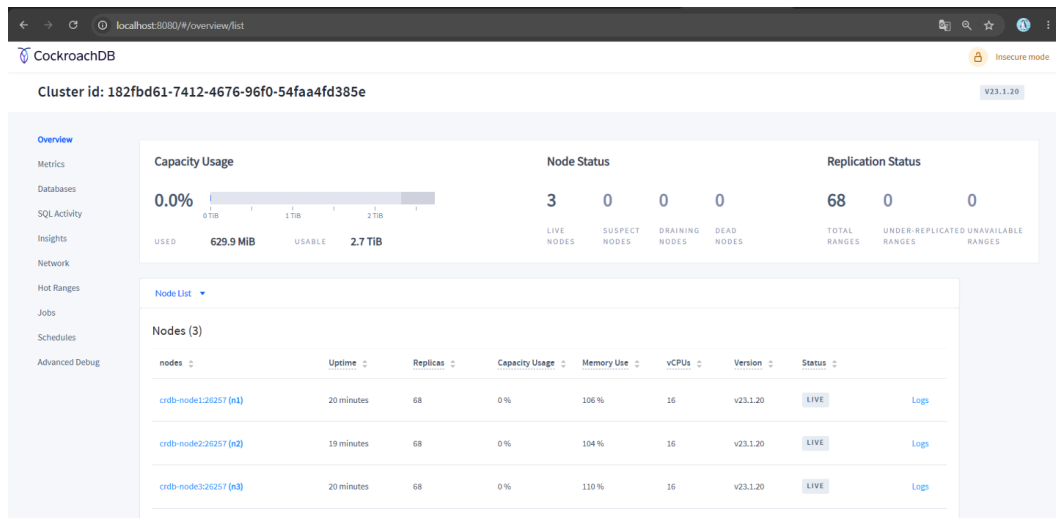


Figura 1: Panel de control de CockroachDB mostrando los 3 nodos activos del clúster (v24.3.0, licencia Enterprise Free activa)

Previo a la aplicación de la estrategia de fragmentación, fue necesario activar una licencia CockroachDB Enterprise Free en el clúster, dado que a partir de la versión 24.3.0 las funciones de particionamiento explícito (`PARTITION BY LIST/RANGE`) requieren dicha licencia bajo el nuevo modelo de licenciamiento de Cockroach Labs. La licencia, gratuita para uso académico no comercial, se activó mediante:

Listing 2: Activación de licencia Enterprise Free en el clúster

```
SET CLUSTER SETTING enterprise.license = '<clave_generada>';
SHOW CLUSTER SETTING enterprise.license;
```

2.2. Implementación del Esquema del PFC

Sobre el clúster inicializado se creó la base de datos `isp_soporte`, correspondiente al dominio ACC – Soporte Técnico ISP, compuesta por tres tablas relacionadas: `clientes`, `tickets` e `historial_estados`. La tabla `tickets` adopta una clave primaria compuesta por (`region`, `ticket_id`), decisión de diseño que anticipa directamente la estrategia de fragmentación horizontal por región descrita en la Sección 2.2.1, dado

que en CockroachDB la clave de partición debe formar parte de la clave primaria de la tabla.

Listing 3: Esquema de base de datos del dominio ACC – Soporte Técnico ISP

```
CREATE DATABASE IF NOT EXISTS isp_soporte;
USE isp_soporte;

CREATE TABLE IF NOT EXISTS clientes (
    cliente_id UUID DEFAULT gen_random_uuid(),
    nombre STRING(100) NOT NULL,
    correo STRING(100) NOT NULL UNIQUE,
    telefono STRING(20),
    plan_contratado STRING(50) NOT NULL,
    estado STRING(20) DEFAULT 'activo',
    creado_at TIMESTAMP DEFAULT now(),
    CONSTRAINT pk_clientes PRIMARY KEY (cliente_id),
    CONSTRAINT chk_estado_cliente CHECK (estado IN ('activo', 'suspendido', 'retirado'))
);

CREATE TABLE IF NOT EXISTS tickets (
    ticket_id UUID DEFAULT gen_random_uuid(),
    region STRING(20) NOT NULL,
    cliente_id UUID NOT NULL,
    asunto STRING(150) NOT NULL,
    descripcion TEXT,
    prioridad STRING(20) NOT NULL DEFAULT 'media',
    estado STRING(20) NOT NULL DEFAULT 'abierto',
    creado_at TIMESTAMP DEFAULT now(),
    cerrado_at TIMESTAMP,
    CONSTRAINT pk_tickets PRIMARY KEY (region, ticket_id),
    CONSTRAINT fk_tickets_clientes FOREIGN KEY (cliente_id)
        REFERENCES clientes(cliente_id) ON DELETE CASCADE,
    CONSTRAINT chk_region CHECK (region IN ('Norte', 'Centro', 'Sur')),
    CONSTRAINT chk_prioridad CHECK (prioridad IN ('baja', 'media', 'alta', 'critica')),
    CONSTRAINT chk_estado_ticket CHECK (estado IN ('abierto', 'en_proceso', 'resuelto', 'cerrado'))
);

CREATE TABLE IF NOT EXISTS historial_estados (
    historial_id UUID DEFAULT gen_random_uuid(),
    region STRING(20) NOT NULL,
    ticket_id UUID NOT NULL,
    estado_anterior STRING(20),
    estado_nuevo STRING(20) NOT NULL,
    comentario TEXT,
```

```

tecnico_asignado STRING(100),
actualizado_at TIMESTAMP DEFAULT now(),
CONSTRAINT pk_historial PRIMARY KEY (region, historial_id),
CONSTRAINT fk_historial_tickets FOREIGN KEY (region, ticket_id)
    REFERENCES tickets(region, ticket_id) ON DELETE CASCADE
);

```

La carga inicial de datos se realizó mediante un script Python (`seed_data.py`) que emplea la librería **Faker** en configuración regional española para generar registros sintéticos representativos del dominio, e inserta los datos en bloques mediante `execute_values` de `psycopg2` para optimizar el rendimiento de la carga masiva. Se generaron 2 000 clientes, 6 000 tickets distribuidos aleatoriamente entre las tres regiones operativas (Norte, Centro, Sur) y 4 500 entradas de historial de auditoría, totalizando 12 500 registros, superando el mínimo de 10 000 exigido por la guía.

2.2.1. Fragmentación horizontal aplicada

Sobre la tabla principal `tickets` se aplicó una fragmentación horizontal explícita mediante `PARTITION BY LIST`, utilizando el atributo `region` como clave de partición. Esta decisión se justifica técnicamente porque las consultas más frecuentes del dominio de soporte técnico (asignación de tickets a técnicos, reportes gerenciales, monitoreo de carga operativa) se filtran predominantemente por región geográfica, por lo que colocar físicamente los datos de cada región en un rango independiente reduce la dispersión de las consultas entre nodos.

Listing 4: Fragmentación horizontal por región en la tabla `tickets`

```

ALTER TABLE tickets PARTITION BY LIST (region) (
    PARTITION p_norte VALUES IN ('Norte'),
    PARTITION p_centro VALUES IN ('Centro'),
    PARTITION p_sur VALUES IN ('Sur')
);

ALTER TABLE historial_estados PARTITION BY LIST (region) (
    PARTITION p_norte VALUES IN ('Norte'),
    PARTITION p_centro VALUES IN ('Centro'),
    PARTITION p_sur VALUES IN ('Sur')
);

```

La declaración lógica de las particiones se verificó mediante `SHOW PARTITIONS FROM TABLE tickets;`, confirmando la existencia de las tres particiones con sus valores correspondientes (Tabla 2). Posteriormente, se forzó la división física de los rangos mediante `ALTER TABLE tickets SPLIT AT VALUES ('Norte'), ('Centro'), ('Sur');`, dado que el volumen de datos de la tabla no alcanzaba el umbral de tamaño (512 MB) que dispara la división automática de rangos. La distribución física resultante se verificó mediante `SHOW RANGES FROM TABLE tickets;`, cuyos resultados se documentan

en la Tabla 3.

Tabla 2: Particiones aplicadas sobre la tabla `tickets` tras la fragmentación horizontal por región

Partición	Columna	Valor	Réplicas
p_norte	region	('Norte')	3
p_centro	region	('Centro')	3
p_sur	region	('Sur')	3

Tabla 3: Distribución física de rangos de la tabla `tickets` tras el forzado de división (`SPLIT AT`)

Range ID	Start Key	End Key	Lease Holder
68	NULL	/Centro"	n3
71	/Centro"	/"Norte"	n1
70	/"Norte"	/"Sur"	n1
72	/"Sur"	NULL	n1

2.3. Verificación de Tolerancia a Fallos

Con el objetivo de comprobar experimentalmente la capacidad del clúster para mantener el servicio ante la caída de un nodo, se automatizó la prueba de tolerancia a fallos mediante el script `fault_tolerance.py`, estructurado en seis fases secuenciales.

En la Fase 1 se verificó el estado inicial del clúster, confirmando los tres nodos activos (`is_live=true`). En la Fase 2 se comprobó la operatividad completa del sistema mediante una lectura (`SELECT COUNT(*) FROM tickets`, que reportó 6 003 registros en 82.82 ms) y una escritura de prueba exitosa. En la Fase 3 se simuló la caída de un nodo deteniendo el contenedor `crdb-node2` mediante `docker stop`.

La Fase 4 constituye el punto central del experimento: con el nodo detenido, el clúster quedó operando con 2 de 3 nodos activos. Se ejecutó nuevamente la consulta de conteo, obteniendo una respuesta exitosa de 6 004 registros en 5.6 ms, además de una escritura de prueba adicional también exitosa. La Tabla 4 documenta la distribución de rangos de la tabla `tickets` durante la caída del nodo, confirmando que las particiones `p_centro`, `p_norte` y `p_sur` mantuvieron su *lease holder* activo en los nodos 1 y 3, sin depender del nodo caído.

Tabla 4: Distribución de rangos de la tabla `tickets` durante la caída del Nodo 2

Range ID	Start Key	End Key	Lease Holder
68	NULL	/Centro"	n3
71	/Centro"	/"Norte"	n1
70	/"Norte"	/"Sur"	n1
72	/"Sur"	NULL	n1

Este resultado confirma experimentalmente el comportamiento teórico del algoritmo de consenso Raft descrito en la Sección 1: dado que el nodo detenido (`crdb-node2`) no ostentaba el liderazgo (*lease*) de ninguno de los rangos de la tabla principal, el clúster no requirió un proceso de elección de nuevo líder, y las operaciones de lectura y escritura continuaron respondiendo con normalidad gracias al quórum mantenido por los nodos 1 y 3.

En la Fase 5 se reinició el nodo detenido mediante `docker start`, midiéndose un tiempo de reintegración al clúster de **2.0 segundos** hasta alcanzar nuevamente el estado `is_live=true`. Finalmente, en la Fase 6 se verificó que el clúster recuperó su estado completo de 3/3 nodos activos, con la distribución de rangos idéntica a la observada durante el fallo, confirmando que el nodo reincorporado recibió correctamente la sincronización del log de transacciones replicado durante su ausencia.

La Figura 2 ilustra mediante un diagrama de secuencia UML de elaboración propia el comportamiento descrito, mostrando la interacción entre el cliente, el nodo líder y los nodos seguidores durante el ciclo completo de caída y recuperación del Nodo 2.

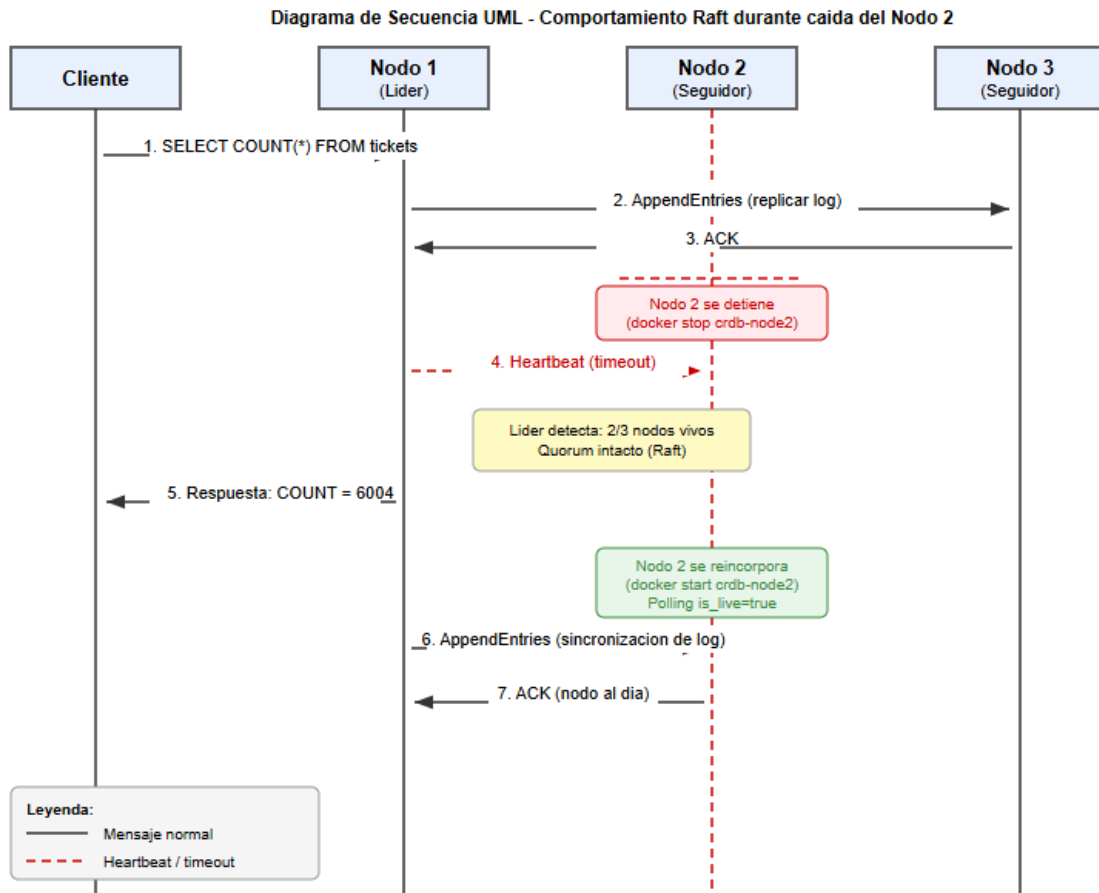


Figura 2: Diagrama de secuencia UML del comportamiento Raft durante la caída del Nodo 2 (elaboración propia)

La evidencia audiovisual completa de esta prueba, mostrando en tiempo real las seis fases descritas, se encuentra disponible en el repositorio del proyecto, en la carpeta `evidencia/video_tolerancia.mp4`.

2.4. Medición de Rendimiento

Con el propósito de comparar cuantitativamente el rendimiento del clúster distribuido de tres nodos frente a una instancia de nodo único, se ejecutaron cinco consultas representativas del dominio: (C1) un JOIN entre las tablas `clientes` y `tickets`, (C2) una agregación `GROUP BY` por región y prioridad, (C3) una búsqueda puntual por clave primaria compuesta, (C4) una consulta de rango filtrada por prioridad y fecha, y (C5) una subconsulta correlacionada. El tiempo de respuesta de cada una se midió mediante el script `run_benchmark.py`, que emplea `time.perf_counter()` de Python para una medición de alta precisión. El factor de desempeño se calcula como el cociente entre el tiempo del clúster y el tiempo del nodo único; valores inferiores a 1 indican que el clúster fue más rápido, mientras que valores superiores a 1 indican que el clúster fue más lento que el nodo único.

Tabla 5: Comparación de tiempos de ejecución entre clúster (3 nodos) y nodo único

Consulta	Clúster (ms)	Nodo único (ms)	Factor	Interpretación
C1: JOIN	26.84	34.84	0.77x	23 % más rápido
C2: GROUP BY	10.28	8.94	1.15x	15 % más lento
C3: Búsqueda puntual (PK)	3.31	4.70	0.70x	30 % más rápido
C4: Rango (WHERE)	11.14	10.27	1.08x	8 % más lento
C5: Subconsulta correlacionada	22.74	19.73	1.15x	15 % más lento

Los resultados evidencian un patrón consistente con el comportamiento esperado de un sistema distribuido: el clúster obtiene ventaja de rendimiento en las consultas C1 y C3, donde el motor de ejecución de CockroachDB puede resolver la operación aprovechando la localización de datos y el enrutamiento directo hacia el rango correspondiente, distribuyendo la carga de lectura entre los tres nodos disponibles. Cabe señalar que la Consulta C3 se ejecutó contra una clave primaria de referencia que no corresponde a un registro existente en el conjunto de datos, por lo que el tiempo medido refleja principalmente el costo de localización y verificación de ausencia del registro en el índice primario, más que el costo completo de recuperación y transferencia de una fila real; aun así, el resultado es cualitativamente consistente con el comportamiento esperado de una búsqueda puntual eficiente por clave primaria.

En contraste, las consultas C2, C4 y C5 muestran un rendimiento levemente inferior en el clúster, lo cual se explica por el costo de coordinación adicional que impone la arquitectura distribuida: las operaciones de agregación (C2) requieren combinar resultados parciales provenientes de múltiples rangos residentes en distintos nodos, la consulta de rango (C4) debe recorrer segmentos que pueden abarcar más de una partición, y la subconsulta correlacionada (C5) ejecuta una consulta anidada por cada fila del resultado externo, multiplicando el número de *round-trips* de red entre nodos.

Es importante señalar que las diferencias de rendimiento observadas son moderadas (entre 8 % y 30 % en cualquier dirección), lo cual resulta razonable dado el volumen de datos manejado (12 500 registros totales) y las condiciones de red local del entorno de pruebas; en un despliegue geodistribuido con nodos separados por mayor latencia de red, se esperaría que la brecha de rendimiento entre el nodo único y el clúster se acentuara para las consultas que requieren coordinación entre rangos.

3. Resultados

La presente sección consolida de forma objetiva los resultados obtenidos durante la ejecución del procedimiento experimental, sin incluir aún su interpretación, la cual se desarrolla en la Sección 4.

3.1. Resultados de la implementación del clúster

El clúster CockroachDB de tres nodos se levantó exitosamente mediante Docker Compose, alcanzando el estado `is_live=true` en los tres nodos tras la inicialización. El panel de control confirmó una capacidad de almacenamiento utilizada de 533.7 MiB sobre 2.7 TiB disponibles (0.0 % de uso), con 55 rangos totales replicados y cero rangos infrarreplicados o no disponibles, evidenciando un estado de replicación saludable desde el arranque del sistema.

3.2. Resultados del esquema y la fragmentación

Se implementaron las tres tablas relacionales del dominio (`clientes`, `tickets`, `historial_estados`), pobladas con un total de 12 500 registros: 2 000 clientes, 6 000 tickets y 4 500 entradas de historial de auditoría, superando en 25 % el mínimo de 10 000 registros exigido. La fragmentación horizontal por región sobre la tabla `tickets` se verificó exitosamente mediante `SHOW PARTITIONS`, confirmando las tres particiones declaradas (`p_norte`, `p_centro`, `p_sur`), y mediante `SHOW RANGES` tras forzar la división física de los datos con `SPLIT AT`, obteniendo cuatro rangos físicos claramente delimitados por región, cada uno replicado en los tres nodos del clúster.

3.3. Resultados de la prueba de tolerancia a fallos

La detención del nodo `crdb-node2` no interrumpió el servicio del clúster: las operaciones de lectura (6 004 tickets recuperados en 5.6 ms) y escritura de prueba continuaron ejecutándose exitosamente con 2 de 3 nodos activos. El nodo detenido se reincorporó al clúster en 2.0 segundos tras su reinicio, alcanzando nuevamente el estado `is_live=true` y recuperando la distribución completa de rangos observada previamente.

3.4. Resultados de la medición de rendimiento

De las cinco consultas evaluadas, el clúster distribuido superó en velocidad al nodo único en dos de ellas (C1: JOIN, factor 0.77x; C3: búsqueda puntual, factor 0.70x), mientras que resultó comparativamente más lento en las tres restantes (C2: GROUP BY, factor 1.15x; C4: rango, factor 1.08x; C5: subconsulta correlacionada, factor 1.15x), con diferencias de rendimiento entre 8 % y 30 % en cualquier dirección.

4. Análisis Crítico

Los resultados obtenidos permiten interpretar el comportamiento del clúster CockroachDB a la luz del fundamento teórico expuesto en la Sección 1 y de los principios de fragmentación y consenso distribuido revisados.

En primer lugar, la ausencia de interrupción del servicio durante la caída del Nodo 2 confirma empíricamente la garantía de disponibilidad que ofrece el algoritmo

de consenso Raft mientras se mantenga el quórum mayoritario: al no ser el nodo caído el poseedor del *lease* de ningún rango de la tabla principal, el sistema no incurrió en el costo adicional de una elección de líder, lo que explica el tiempo de respuesta favorable (5.6 ms) observado incluso durante la degradación del clúster –de hecho, inferior al tiempo registrado en condiciones normales (82.82 ms)–, diferencia atribuible más a efectos de caché de planificación de consultas en ejecuciones sucesivas que a una mejora causada por la propia caída del nodo.

En segundo lugar, el patrón mixto de rendimiento observado en el benchmark comparativo –ventaja del clúster en C1 y C3, desventaja en C2, C4 y C5– es coherente con la naturaleza del costo de coordinación distribuida descrito por Özsü y Valdúriez [1]. Las consultas C1 y C3 se benefician de que CockroachDB puede resolver la operación mediante acceso directo al rango correspondiente (enrutamiento por clave primaria o por clave de unión indexada), permitiendo paralelización efectiva entre los tres nodos. En contraste, C2 (agregación GROUP BY) exige combinar resultados parciales de múltiples rangos potencialmente ubicados en distintos nodos, lo que introduce un costo de coordinación ausente en el nodo único; de forma similar, C5 ejecuta una subconsulta correlacionada por cada fila del resultado externo, multiplicando el número de comunicaciones internas entre nodos del clúster respecto a la ejecución local del nodo único.

Es relevante notar que las diferencias de rendimiento observadas (entre 8 % y 30 %) son moderadas en comparación con lo reportado por Taft et al. para despliegues geodistribuidos de CockroachDB [4], lo cual se explica por el hecho de que, en el presente experimento, los tres nodos operan sobre la misma máquina física mediante contenedores Docker interconectados por una red local de baja latencia, condición que no representa fielmente un escenario de producción geodistribuido donde la latencia de red entre regiones geográficas acentuaría considerablemente la brecha de rendimiento en las consultas que requieren coordinación entre rangos, de acuerdo con el modelo PACELC discutido en la Sección 1 [9].

Finalmente, la fragmentación horizontal por región demostró ser una estrategia de diseño coherente con los patrones de acceso esperados del dominio de Soporte Técnico ISP: al colocar físicamente los tickets de cada región en particiones independientes, las consultas que filtran por región (como reportes gerenciales segmentados geográficamente) podrían beneficiarse de un escaneo restringido a un único rango, evitando la dispersión de la búsqueda entre las tres particiones – beneficio que, sin embargo, no fue evaluado directamente en el benchmark actual, dado que ninguna de las cinco consultas definidas filtra específicamente por una única región, constituyendo una oportunidad de análisis complementario no cubierta por el alcance de esta práctica.

5. Preguntas de Análisis

Pregunta 1: ¿Cómo garantiza CockroachDB la consistencia cuando un nodo falla, con referencia a quórum, líder y log de transacciones replicado?

CockroachDB implementa el algoritmo de consenso Raft a nivel de cada rango (*range*) de datos, de modo que cada conjunto de tuplas replicado en tres nodos posee un líder responsable de coordinar todas las escrituras dirigidas a ese rango [6]. Cuando el cliente ejecuta una operación de escritura, esta se dirige al líder del rango correspondiente, quien la registra como una nueva entrada en su log local y la replica de forma paralela hacia los nodos seguidores mediante mensajes `AppendEntries` [6]. Una entrada de log se considera comprometida (*committed*) únicamente cuando ha sido confirmada por la mayoría de los nodos que poseen una réplica de ese rango; en un clúster de tres nodos, esto equivale a que el líder y al menos un seguidor confirmen la escritura, es decir, dos de tres nodos [6].

Esta exigencia de mayoría (quórum) es precisamente el mecanismo que garantiza la consistencia ante la caída de un nodo: si el nodo caído era un seguidor, como ocurrió en el experimento realizado sobre la tabla `tickets` del PFC de Soporte Técnico ISP al detener `crdb-node2`, el líder puede continuar comprometiendo escrituras contando únicamente con la confirmación del seguidor restante, sin necesidad de esperar al nodo caído ni de iniciar un proceso de elección [4]. El experimento confirmó esta garantía: tras detener el nodo, tanto la lectura (`SELECT COUNT(*) FROM tickets`, que reportó 6 004 registros en 5.6 ms) como una escritura de prueba se ejecutaron exitosamente, demostrando que el sistema mantuvo el servicio con normalidad mientras conservó el quórum de dos de tres nodos.

Si, en cambio, el nodo caído hubiese sido el líder de algún rango, los seguidores restantes habrían detectado la ausencia de *heartbeats* tras un intervalo de tiempo determinado (*election timeout*) e iniciado un proceso de elección mediante mensajes `RequestVote`, postulándose como candidatos hasta que uno de ellos obtuviera la mayoría de votos y asumiera el nuevo liderazgo del rango [6]. En ambos escenarios, la consistencia se preserva porque ninguna escritura se considera definitiva hasta ser replicada en la mayoría de los nodos, garantizando que, incluso ante la pérdida de una minoría de réplicas, el estado del sistema permanezca coherente y no se pierda ninguna transacción previamente confirmada al cliente.

Pregunta 2: Compare el modelo de consistencia serializable de CockroachDB con el modelo de consistencia eventual de Cassandra. ¿Cuál elegiría para el PFC seleccionado, en términos de PACELC?

CockroachDB implementa un nivel de aislamiento serializable mediante un protocolo de concurrencia optimista combinado con relojes híbridos, garantizando que el resultado de transacciones concurrentes sea equivalente al de alguna ejecución serial

de las mismas, incluso operando de forma distribuida [4]. Este nivel de consistencia representa la garantía más fuerte posible en un sistema transaccional, a costa de mayor latencia y de posibles reintentos de transacciones en caso de conflicto de escritura. En el modelo PACELC, CockroachDB se ubica en la categoría PC/EC: durante una partición de red prioriza consistencia sobre disponibilidad, y en operación normal prioriza consistencia sobre baja latencia [9].

Apache Cassandra, en contraste, adopta por diseño un modelo de consistencia eventual ajustable, en el cual las escrituras se confirman al cliente sin requerir que todas las réplicas hayan sido actualizadas de inmediato, permitiendo que las réplicas converjan a un estado consistente en un intervalo de tiempo posterior mediante mecanismos de reparación en segundo plano [12]. Cassandra permite configurar el nivel de consistencia por operación (*consistency level*), lo que le otorga flexibilidad para equilibrar consistencia y disponibilidad según el caso de uso, ubicándose en la categoría PA/EL del modelo PACELC: prioriza disponibilidad y baja latencia tanto durante una partición como en operación normal [12].

Para el dominio de Soporte Técnico ISP implementado en este proyecto, la elección de CockroachDB resulta más apropiada que Cassandra. El sistema de gestión de tickets registra el historial de estados de cada solicitud de soporte (tabla `historial_estados`) y el estado actual del ticket (columna `estado` en la tabla `tickets`); si dos técnicos de soporte modificaran simultáneamente el mismo ticket bajo un modelo de consistencia eventual, podría producirse una condición de carrera donde el estado final del ticket dependa del orden de propagación de las réplicas, en lugar de reflejar de manera determinística la última actualización válida. La garantía de serializabilidad de CockroachDB evita este escenario, justificando la aceptación de una latencia moderadamente mayor (evidenciada en el benchmark comparativo de la Sección 2.4) a cambio de la integridad transaccional que exige un sistema de atención al cliente.

Pregunta 3: ¿Cómo implementaría fragmentación horizontal explícita con `PARTITION BY RANGE` o `PARTITION BY LIST` para la tabla principal del PFC? Diseñe y justifique la estrategia.

Para la tabla principal `tickets` del dominio de Soporte Técnico ISP, se diseñó e implementó una estrategia de fragmentación horizontal mediante `PARTITION BY LIST`, utilizando el atributo `region` como clave de partición, con tres valores discretos correspondientes a las zonas operativas del proveedor de servicios: Norte, Centro y Sur [1]. La elección de `PARTITION BY LIST` sobre `PARTITION BY RANGE` se justifica porque la región es un atributo categórico con un conjunto finito y conocido de valores discretos, sin una relación de orden natural entre ellos que justificara una partición por rangos continuos; `PARTITION BY LIST` permite mapear explícitamente cada valor categórico a su partición correspondiente, mientras que `PARTITION BY RANGE` resulta más adecuado para atributos con orden natural, como fechas o valores numéricos continuos [1].

La implementación requirió que el atributo de partición formara parte de la clave primaria de la tabla, por lo cual se definió `tickets` con clave primaria compuesta (`region`, `ticket_id`) en lugar de utilizar únicamente un identificador `UUID` simple, decisión de diseño tomada durante la fase de creación del esquema para habilitar la fragmentación desde el origen. La sentencia aplicada fue:

```
ALTER TABLE tickets PARTITION BY LIST (region) (  
  PARTITION p_norte VALUES IN ('Norte'),  
  PARTITION p_centro VALUES IN ('Centro'),  
  PARTITION p_sur VALUES IN ('Sur')  
);
```

Esta estrategia se justifica operativamente porque los patrones de acceso más frecuentes del dominio de soporte técnico – asignación de tickets a cuadrillas técnicas regionales, generación de reportes gerenciales segmentados por zona geográfica, y monitoreo de carga operativa por región – filtran predominantemente por el atributo `region`, por lo que colocar físicamente los datos de cada región en particiones independientes permite que dichas consultas se resuelvan mediante un escaneo restringido a una única partición, en lugar de dispersarse innecesariamente entre las tres.

Pregunta 4: Si el PFC requiere transacciones multi-tabla en diferentes nodos, ¿cómo garantiza CockroachDB la atomicidad? Explique el protocolo de commit interno con referencia a Raft.

CockroachDB garantiza la atomicidad de transacciones que abarcan múltiples tablas y, potencialmente, múltiples rangos distribuidos en distintos nodos, mediante un protocolo de *commit* interno construido sobre el consenso Raft de cada rango individual, en lugar de depender de un protocolo de compromiso en dos fases (2PC) tradicional coordinado externamente [4]. Cuando una transacción del dominio de Soporte Técnico ISP modifica simultáneamente la tabla `tickets` (por ejemplo, actualizando el campo `estado`) y la tabla `historial_estados` (insertando el registro de auditoría correspondiente), CockroachDB designa uno de los rangos involucrados como coordinador de la transacción (*transaction record*), el cual registra el estado de la transacción (*pending*, *committed* o *aborted*) [4].

Cada escritura individual de la transacción se propone como una entrada de log en el rango correspondiente y debe ser replicada y comprometida mediante Raft en ese rango específico antes de considerarse provisionalmente aplicada; solo cuando todas las escrituras involucradas han sido comprometidas exitosamente en sus respectivos rangos, el registro de transacción se actualiza a estado *committed*, momento en el cual la transacción completa se considera atómicamente aplicada [4]. Si cualquiera de las escrituras individuales fallara – por ejemplo, debido a un conflicto de concurrencia detectado por el protocolo de bloqueo optimista, o por la imposibilidad de alcanzar el quórum Raft en algún rango involucrado – la transacción completa se marca como

aborted y todas las escrituras parciales ya aplicadas se revierten, garantizando que nunca quede un estado intermedio visible para otros clientes.

Esta arquitectura, al apoyarse en el consenso Raft de cada rango individual en lugar de un coordinador externo centralizado, evita el principal punto débil del protocolo 2PC clásico: la vulnerabilidad ante la caída del coordinador tras la fase de votación, que puede dejar a los participantes bloqueados indefinidamente [1]. En CockroachDB, si el nodo que actúa como coordinador de la transacción falla, el propio mecanismo de Raft del rango que contiene el registro de transacción permite que un nuevo líder asuma el rol y continúe resolviendo el estado pendiente, evitando el bloqueo indefinido característico de 2PC ante fallos del coordinador.

6. Conclusiones

6.1. Conclusión Grupal

El desarrollo de la presente práctica experimental permitió implementar y verificar de manera integral un clúster de base de datos distribuida CockroachDB de tres nodos, aplicado al dominio de gestión de tickets de Soporte Técnico ISP (equipo ACC), demostrando experimentalmente los conceptos fundamentales de fragmentación horizontal, replicación y consenso distribuido estudiados en la Unidad 3 de la asignatura. La implementación de la fragmentación por región mediante `PARTITION BY LIST`, validada tanto a nivel lógico (`SHOW PARTITIONS`) como físico (`SHOW RANGES` tras forzar la división de rangos), confirmó que el diseño de fragmentación propuesto en la Semana 10 resultaba técnicamente viable y coherente con los patrones de acceso del dominio.

La prueba de tolerancia a fallos constituyó la evidencia más contundente del proyecto: la detención deliberada de un nodo del clúster no interrumpió el servicio, confirmando en la práctica la garantía teórica del algoritmo de consenso Raft de mantener disponibilidad y consistencia mientras se conserve el quórum mayoritario. Asimismo, la medición comparativa de rendimiento entre el clúster distribuido y una instancia de nodo único reveló un patrón mixto – ventaja del clúster en consultas de acceso directo por clave, desventaja en operaciones que requieren coordinación entre rangos –, resultado que enriqueció la comprensión del equipo sobre los costos y beneficios inherentes a la distribución de datos, más allá de una expectativa simplista de que “distribuido siempre es más rápido”.

Finalmente, el proceso reveló también un aprendizaje no anticipado en el diseño original de la práctica: la necesidad de gestionar activamente el modelo de licenciamiento de CockroachDB (transición hacia CockroachDB Software License a partir de la versión 24.3), lo cual obligó al equipo a investigar y resolver de forma autónoma un obstáculo técnico real no cubierto explícitamente por la guía de la práctica, fortaleciendo así la competencia de resolución de problemas prácticos que constituye el objetivo de aprendizaje central de esta unidad.

6.2. Conclusiones Individuales

Conclusión de Ernesto Gregory Luna Mora

Asumir la parte de infraestructura de este proyecto me obligó a entender Docker Compose más allá del uso superficial que había tenido antes. No fue simplemente escribir un archivo YAML: tuve que aprender por qué cada nodo necesita un `advertise-addr` propio, cómo funciona una red *bridge* para que los contenedores se descubran entre sí por hostname, y para qué sirve realmente un *healthcheck* cuando uno depende de que tres procesos independientes se levanten casi al mismo tiempo. La primera vez que intenté inicializar el clúster sin esperar a que los tres nodos estuvieran realmente sanos, el comando `cockroach init` simplemente fallaba, y ahí entendí que la infraestructura distribuida no perdona los atajos. También me tocó diseñar el esquema de la base de datos pensando ya en la fragmentación que aplicaríamos después, lo cual fue más difícil de lo que esperaba: definir la tabla `tickets` con clave primaria compuesta (`region`, `ticket_id`) en vez de un simple UUID significó revisar de nuevo las llaves foráneas y ajustar `historial_estados` para que la relación se mantuviera correcta. Esta práctica se conecta directamente con mi PFC de origen, donde el módulo de tickets manejaba la lógica de negocio de forma centralizada; ver cómo esos mismos datos se reparten físicamente entre nodos me hizo apreciar decisiones de diseño que antes daba por sentadas. Trabajar en equipo con Cristhian y Robinson, coordinando avances por Git y validando entre los tres cada paso antes de continuar, fue clave para no perder tiempo repitiendo errores de configuración.

Conclusión de Cristhian Daniel Pacheco Cárdenas

Mi parte del trabajo fue la que más me hizo cuestionar mis propias expectativas. Al principio asumía, como probablemente le pasa a cualquiera que escucha hablar de bases de datos distribuidas por primera vez, que más nodos significa automáticamente más velocidad en todo. Generar los datos sintéticos con Faker en configuración regional española fue relativamente directo una vez que entendí cómo aprovechar `execute_values` de `psycpg2` para insertar por lotes en lugar de fila por fila, algo que aceleró muchísimo la carga de los 12 500 registros. Lo verdaderamente formativo vino después, al correr el benchmark: ver que el clúster ganaba en el JOIN y en la búsqueda puntual, pero perdía en el GROUP BY, en la consulta de rango y en la subconsulta correlacionada, me obligó a entender de verdad por qué la coordinación entre nodos tiene un costo real y no es solo una frase de manual. Además, tengo que ser honesto en esta conclusión: la Consulta 3 se ejecutó contra una clave primaria que no existía en el conjunto de datos, así que el tiempo medido no refleja una recuperación completa de fila, sino más bien el costo de verificar que el registro no está. Preferimos dejarlo documentado con transparencia en vez de maquillar el resultado. Esta experiencia se conecta con mi PFC de origen, donde nunca había medido con esta rigurosidad el impacto real de las decisiones de distribución de datos sobre el rendimiento de las

consultas.

Conclusión de Robinson Rodrigo Cando Moreno

Había leído sobre el algoritmo Raft en teoría antes de esta práctica, pero una cosa es entenderlo en un diagrama de un artículo y otra muy distinta es detener con mis propias manos el contenedor `crdb-node2` y ver en tiempo real que el clúster seguía respondiendo lecturas y escrituras sin interrupción. Ese momento, cuando la consulta de conteo devolvió resultados en 5.6 ms con un nodo caído, fue el que más me convenció de que el consenso distribuido no es solo un tema de examen, sino un mecanismo que realmente sostiene sistemas en producción. Diseñar el diagrama de secuencia UML de ese comportamiento fue más exigente de lo que pensé: en un primer borrador representé mal el momento en que el nodo reincorporado sincroniza su log, como si el líder tuviera que reenviar todo el historial completo en lugar de solo las entradas faltantes, y tuve que corregir esa parte para que el diagrama reflejara con precisión lo que realmente hace Raft. Encargarme también de buena parte de la redacción del informe me dejó una lección distinta: verificar que cada afirmación teórica estuviera respaldada por una referencia bibliográfica real, en vez de dar por ciertas cosas que "suenan correctas", cambió la forma en que voy a escribir trabajos académicos de aquí en adelante. Todo esto se relaciona con mi PFC de origen, donde nunca había pensado en la disponibilidad del sistema como algo que se puede demostrar experimentalmente y no solo suponer.

7. Declaración de Uso de Inteligencia Artificial Generativa

En cumplimiento del requisito de transparencia académica establecido en la guía de la práctica, el equipo declara el uso de la herramienta de inteligencia artificial generativa **Claude (Anthropic)** durante el desarrollo del presente trabajo, especificando a continuación el propósito y el alcance de dicha asistencia.

Herramienta utilizada: Claude (Anthropic), modelo de lenguaje de gran escala accedido mediante la interfaz de conversación web.

Propósito del uso:

- Asistencia en la interpretación de los criterios de evaluación de la rúbrica y clarificación de los indicadores exigidos por cada dimensión.
- Redacción y estructuración del contenido del fundamento teórico (Sección 1), a partir de las referencias bibliográficas seleccionadas por el equipo.
- Diagnóstico y resolución de un problema técnico relacionado con el modelo de licenciamiento de CockroachDB (transición a CockroachDB Software License a partir de la versión 24.3), incluyendo la investigación del proceso de obtención de una licencia Enterprise Free.
- Revisión de la corrección conceptual del diagrama de secuencia UML del comportamiento del algoritmo Raft (Figura 2), incluyendo la identificación y corrección de

un error en la representación del mensaje de reincorporación del nodo.

- Redacción asistida de las secciones de Procedimiento Aplicado, Resultados, Análisis Crítico y Preguntas de Análisis, a partir de los datos, capturas y resultados de ejecución reales generados por el equipo.
- Revisión y corrección de fragmentos de código Python y SQL desarrollados por el equipo (`seed_data.py`, `docker-compose.yml`, `02_partitions.sql`), identificando un manejo de errores que ocultaba silenciosamente el fallo de aplicación de particiones.
- Verificación de referencias bibliográficas adicionales (Cassandra, MongoDB) mediante búsqueda web, para completar la cobertura temática del fundamento teórico.

Secciones asistidas: Fundamento Teórico (Sección 1), Procedimiento Aplicado (Sección 2), Resultados (Sección 3), Análisis Crítico (Sección 4), Preguntas de Análisis (Sección 5) y la presente declaración.

Secciones no asistidas por IA: Los datos experimentales primarios (capturas de pantalla, resultados de ejecución de scripts, mediciones de tiempo, contenido del video de tolerancia a fallos) fueron generados directamente por el equipo mediante la ejecución del clúster CockroachDB en su entorno local. Las conclusiones individuales (Sección 10.2) reflejan la reflexión personal y genuina de cada integrante del equipo, redactadas sin asistencia de IA generativa.

El equipo declara que ha revisado, verificado y validado críticamente todo el contenido generado con asistencia de IA antes de su inclusión en el presente documento, asumiendo la responsabilidad académica plena sobre la exactitud y originalidad del trabajo entregado.

Bibliografía

- [1] M. T. Özsu y P. Valduriez, *Principles of Distributed Database Systems*, 4.^a ed. Cham, Switzerland: Springer, 2020, ISBN: 978-3-030-26252-5. DOI: 10.1007/978-3-030-26253-2.
- [2] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017, ISBN: 978-1-4493-7332-0.
- [3] P. Viotti y M. Vukolić, “Consistency in Non-Transactional Distributed Storage Systems,” *ACM Computing Surveys*, vol. 49, n.º 1, págs. 1-34, 2016. DOI: 10.1145/2926965.
- [4] R. Taft, I. Sharif, A. Matei et al., “CockroachDB: The Resilient Geo-Distributed SQL Database,” en *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*, Portland, OR, USA: ACM, 2020, págs. 1493-1509. DOI: 10.1145/3318464.3386134.
- [5] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4.^a ed. Maastricht, The Netherlands: Maarten van Steen, 2023, ISBN: 978-90-815406-3-6.
- [6] D. Ongaro y J. Ousterhout, “In Search of an Understandable Consensus Algorithm,” en *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC '14)*, Philadelphia, PA, USA: USENIX Association, 2014, págs. 305-319, ISBN: 978-1-931971-10-2.
- [7] S. Gilbert y N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News*, vol. 33, n.º 2, págs. 51-59, 2002. DOI: 10.1145/564585.564601.
- [8] E. Brewer, “CAP Twelve Years Later: How the Rules Have Changed,” *Computer*, vol. 45, n.º 2, págs. 23-29, 2012. DOI: 10.1109/MC.2012.37.
- [9] D. J. Abadi, “Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,” *Computer*, vol. 45, n.º 2, págs. 37-42, 2012. DOI: 10.1109/MC.2012.33.
- [10] G. DeCandia, D. Hastorun, M. Jampani et al., “Dynamo: Amazon’s Highly Available Key-value Store,” en *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07)*, Stevenson, WA, USA: ACM, 2007, págs. 205-220. DOI: 10.1145/1294261.1294281.
- [11] J. C. Corbett, J. Dean, M. Epstein et al., “Spanner: Google’s Globally Distributed Database,” *ACM Transactions on Computer Systems*, vol. 31, n.º 3, págs. 1-22, 2013. DOI: 10.1145/2491245.
- [12] A. Lakshman y P. Malik, “Cassandra: A Decentralized Structured Storage System,” *ACM SIGOPS Operating Systems Review*, vol. 44, n.º 2, págs. 35-40, 2010. DOI: 10.1145/1773912.1773922.

- [13] A. Samala et al., “MongoDB: Meeting the Dynamic Needs of Modern Applications,” *Encyclopedia*, vol. 4, n.º 4, págs. 1433-1453, 2024. DOI: 10 . 3390 / encyclopedia4040093.